

Assignment 1, P1. Pacman Search

Total: 150 points

In this problem, taken from the Berkeley AI Pacman project, your Pacman agent will find paths through his maze world, both to reach a particular location and to collect food efficiently. You will build general search algorithms and apply them to Pacman scenarios. This project includes an autograder for you to grade your answers on your machine. This can be run with the command:

```
python autograder.py
```

If you pass a test case, the autograder prints out `*** PASS: ..` with the name of the test case that was used. It will run all tests on all problems and give you a composite score. However, if you want to run the autograder on a specific question, say Question 2, then use

```
python autograder.py -q q2
```

Code: The code for this problem consists of several Python files, some of which you will need to read and understand in order to complete the assignment, and some of which you can ignore. You can download all the code and supporting files in the zip file **assignment1.zip** on Canvas.

Files to Edit and Submit: You will fill in portions of *search.py* and *searchAgents.py* during the assignment. You should submit these files only. Please do not change the other files in this distribution or submit any of our original files other than these files.

Evaluation: Your code will be autograded for technical correctness. Please do not change the names of any provided functions or classes within the code.

Collaboration Policy: Please refer to the course syllabus.

Citation Policy: Please refer to the course syllabus.

Late Policy: Please refer to the course syllabus.

Getting Help: You are not alone! If you find yourself stuck on something, contact the course staff for help on Piazza. Office hours, and Piazza are there for your support; please use them.

Discussion: Please be careful not to post spoilers on Piazza.

Code structure

File Name	Read?	Edit?	Description
search.py	Yes	Yes	Where all of your search algorithms will be.
searchAgents.py	Yes	Yes	Where all of your search-based agents be.
pacman.py	Yes	No	The main file that runs Pacman games. This file describes a Pacman GameState type, which you use in this project.
game.py	Yes	No	The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.
util.py	Yes	No	Useful data structures for implementing search algorithms.
graphicsDisplay.py	No	No	Graphics for Pacman.
graphicsUtils.py	No	No	Support for Pacman graphics.
textDisplay.py	No	No	ASCII graphics for Pacman.
ghostAgents.py	No	No	Agents to control ghosts.
keyboardAgents.py	No	No	Keyboard interfaces to control Pacman.
layout.py	No	No	Code for reading layout files and storing their contents.
autograder.py	No	No	Project autograder.
testParser.py	No	No	Parses autograder test and solution files.
testClasses.py	No	No	General autograding test classes.
test cases/	No	No	Directory containing the test cases for each question.
searchTestClasses.py	No	No	Search algorithm specific autograding test classes

P1.Q1 Warming up

0 Points

After downloading the code, unzipping it, and changing to the directory, you should be able to play a game of Pacman by typing the following at the command line:

```
python pacman.py
```

Pacman lives in a shiny blue world of twisting corridors and tasty round treats. Navigating this world efficiently will be Pacman's first step in mastering his domain.

The simplest agent in *searchAgents.py* is called the *GoWestAgent*, which always goes west (a trivial reflex agent). This agent can occasionally win:

```
python pacman.py --layout testMaze --pacman GoWestAgent
```

But, things get ugly for this agent when turning is required:

```
python pacman.py --layout tinyMaze --pacman GoWestAgent
```

If Pacman gets stuck, you can exit the game by typing CTRL-c into your terminal. Soon, your agent will solve not only tinyMaze, but any maze you want.

Note that pacman.py supports a number of options that can each be expressed in a long way (e.g., --layout) or a short way (e.g., -l). You can see the list of all options and their default values via:

```
python pacman.py -h
```

Also, all of the commands that appear in this project also appear in commands.txt, for easy copying and pasting. In UNIX/Mac OS X, you can even run all these commands in order with

```
bash commands.txt.
```

No submission is required for this question (P1.Q1).

P1.Q2 A* search

25 Points

In *searchAgents.py*, you'll find a fully implemented *SearchAgent*, which plans out a path through Pacman's world and then executes that path step-by-step. The search algorithms for formulating a plan are not implemented -- that's your job. First, test that the *SearchAgent* is working correctly by running:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

The command above tells the *SearchAgent* to use *tinyMazeSearch* as its search algorithm, which is implemented in *search.py*. Pacman should navigate the maze successfully.

Now it's time to write full-fledged generic search functions to help Pacman plan routes! Pseudocode for the search algorithms you'll write can be found in the lecture slides. Remember that a search node must contain not only a state but also the information necessary to reconstruct the path (plan) which gets to that state.

Important note: All of your search functions need to return a list of actions that will lead the agent from the start to the goal. These actions all have to be legal moves (valid directions, no moving through walls).

Important note: Make sure to use the Stack, Queue and PriorityQueue data structures provided to you in *util.py*! These data structure implementations have particular properties which are required for compatibility with the autograder.

Implement A* graph search in the empty function *aStarSearch* in *search.py*. A* takes a heuristic function as an argument. Heuristics take two arguments: a state in the search problem (the main argument), and the problem itself (for reference information). The *nullHeuristic* heuristic function in *search.py* is a trivial example. You can test your A* implementation on the original problem of finding a path through a maze to a fixed position using the Manhattan distance heuristic (implemented already as *manhattanHeuristic* in *searchAgents.py*).

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

The Pacman board will show an overlay of the states explored, and the order in which they were explored (brighter red means earlier exploration).

P1.Q3 Finding all the corners

20 Points

The real power of A* will only be apparent with a more challenging search problem. Now, it's time to formulate a new problem and design a heuristic for it. In corner mazes, there are four dots, one in each corner. Our new search problem is to find the shortest path through the maze that touches all four corners (whether the maze actually has food there or not). Note that for some mazes like *tinyCorners*, the shortest path does not always go to the closest food first!

Hint: the shortest path through *tinyCorners* takes 28 steps.

Note: Make sure to complete the search algorithms before working on this question.

Implement the *CornersProblem* search problem in *searchAgents.py*. You will need to choose a state representation that encodes all the information necessary to detect whether all four corners have been reached. Now, your search agent should solve:

```
python pacman.py -l tinyCorners -p SearchAgent -a fn=astar,prob=CornersProblem
```

```
python pacman.py -l mediumCorners -p SearchAgent -a fn=astar,prob=CornersProblem
```

To receive full credit, you need to define an abstract state representation that does not encode irrelevant information (like the position of ghosts, where extra food is, etc.). In particular, do not use a Pacman GameState as a search state. Your code will be very, very slow if you do (and also wrong).

Hint: The only parts of the game state you need to reference in your implementation are the starting Pacman position and the location of the four corners.

P1.Q4 Heuristic for the Corners Problem

30 Points

Implement a non-trivial, consistent heuristic for the *CornersProblem* in *cornersHeuristic*.

```
python pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

Note: AStarCornersAgent is a shortcut for -p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic.

Admissibility vs. Consistency: Remember, heuristics are just functions that take search states and return numbers that estimate the cost to a nearest goal. More effective heuristics will return values closer to the actual goal costs. To be admissible, the heuristic values must be lower bounds on the actual shortest path cost to the nearest goal (and non negative). To be consistent, it must additionally hold that if an action has cost c , then taking that action can only cause a drop in heuristic of at most c . Remember that admissibility isn't enough to guarantee correctness in graph search -- you need the stronger condition of consistency. However, admissible heuristics are usually also consistent, especially if they are derived from problem relaxations. Therefore it is usually easiest to start out by brainstorming admissible heuristics. Once you have an admissible heuristic that works well, you can check whether it is indeed consistent, too. The only way to guarantee consistency is with a proof. However, inconsistency can often be detected by verifying that for each node you expand, its successor nodes are equal or higher in f -value. Moreover, if UCS (uniform cost search) and A^* ever return paths of different lengths, your heuristic is inconsistent.

Non-Trivial Heuristics: The trivial heuristics are the ones that return zero everywhere (UCS) and the heuristic which computes the true completion cost. The former won't save you any time, while the latter will timeout the autograder. You want a heuristic which reduces total compute time, though for this assignment the autograder will only check node counts (aside from enforcing a reasonable time limit).

Grading: Your heuristic must be a non-trivial non-negative consistent heuristic to receive any points. Make sure that your heuristic returns 0 at every goal state and never returns a negative value. Depending on how few nodes your heuristic expands, you'll be graded:

Number of nodes expanded	Grade
more than 2000	0/30
at most 2000	10/30
at most 1600	20/30
at most 1200	30/30

Remember: If your heuristic is inconsistent, you will receive no credit, so be careful!

P1.Q5: Suboptimal search

25 Points

Sometimes, even with A* and a good heuristic, finding the optimal path through all the dots is hard. In these cases, we'd still like to find a reasonably good path, quickly. In this section, you'll write an agent that always greedily eats the closest dot. *ClosestDotSearchAgent* is implemented for you in *searchAgents.py*, but it's missing a key function that finds a path to the closest dot.

Implement the function *findPathToClosestDot* in *searchAgents.py*. Our agent solves this maze (suboptimally!) in under a second with a path cost of 350:

```
python pacman.py -l bigSearch -p ClosestDotSearchAgent -z .5
```

Hint: The quickest way to complete *findPathToClosestDot* is to fill in the *AnyFoodSearchProblem*, which is missing its goal test. Then, solve that problem with an appropriate search function. The solution should be very short! Your *ClosestDotSearchAgent* won't always find the shortest possible path through the maze. Make sure you understand why and try to come up with a small example where repeatedly going to the closest dot does not result in finding the shortest path for eating all the dots.

P1.Q6: Eating all the dots

50 Points

Now we'll solve a hard search problem: eating all the Pacman food in as few steps as possible. For this, we'll need a new search problem definition which formalizes the food-clearing problem: *FoodSearchProblem* in *searchAgents.py* (implemented for you). A solution is defined to be a path that collects all of the food in the Pacman world. For the present problem, solutions do not take into account any ghosts or power pellets; solutions only depend on the placement of walls, regular food and Pacman. If you have written your general search methods correctly, A* with a null heuristic (equivalent to uniform-cost search, UCS) should quickly find an optimal solution to *testSearch* with no code change on your part (total cost of 7).

```
python pacman.py -l testSearch -p AStarFoodSearchAgent
```

Note: *AStarFoodSearchAgent* is a shortcut for `-p SearchAgent -a fn=astar,prob=FoodSearchProblem,heuristic=foodHeuristic`.

You should find that UCS starts to slow down even for the seemingly simple *tinySearch*. As a reference, our implementation takes 2.5 seconds to find a path of length 27 after expanding 5057 search nodes.

Fill in *foodHeuristic* in *searchAgents.py* with a consistent heuristic for the *FoodSearchProblem*. Try your agent on the *trickySearch* board:

```
python pacman.py -l trickySearch -p AStarFoodSearchAgent
```

The UCS agent finds the optimal solution in about 13 seconds, exploring over 16,000 nodes. Make sure that your heuristic returns 0 at every goal state and never returns a negative value. Depending on how few nodes your heuristic expands, you'll get additional points:

Number of nodes expanded (n)	Grade
more than 18000	0
$8000 \leq n \leq 18000$	$(1 - ((n - 8000) / 10000)) * 50$
less than 8000	50

Remember: If your heuristic is inconsistent, you will receive no credit, so be careful! Can you solve *mediumSearch* in a short time? If so, we're either very, very impressed, or your heuristic is inconsistent.